

Image Processing Practical Project

Image Enhancement and Local Filters

Student Name: _____

Student ID: _____

Submission Date: June 4, 2026

Contents

1	Part One: Contrast Enhancement and Brightness Correction	3
1.0.1	Contrast Enhancement and Brightness Correction	3
1.0.2	Part 1: Implementation (Practical)	3
1.0.3	Part 2: Analysis and Theory	3
2	Part Two: Histogram Equalization	5
2.0.1	Histogram Equalization	5
2.0.2	Requirements	5
2.0.3	Submission Method	5
3	Part Three: Local Filters - Noise Analysis and Evaluation	6
3.0.1	Local Filters	6
3.0.2	Part 1: Implementation and Execution	6
3.0.3	Part 2: Result Analysis	6
3.0.4	Programming Requirements	7
3.0.5	Submission Method	7
4	Summary of Deliverables	8

Submission Guidelines

Please submit a single compressed file (.zip) named as: **IP_StudentName_StudentId.zip**

1 Part One: Contrast Enhancement and Brightness Correction

1.0.1 Contrast Enhancement and Brightness Correction

In digital image processing, one of the fundamental and important methods for improving the visual quality of an image is using spatial domain transformations on pixel intensity values. In this question, you must implement functions for several important transformations and demonstrate your analytical knowledge about their application in real-world scenarios.

This question consists of two parts: Implementation and Theory.

1.0.2 Part 1: Implementation (Practical)

In this part, you must create a file named `enhancement.py` and implement the following three functions. These functions are responsible for modifying the pixel intensity values of the input image according to specified mathematical formulas.

- **Input Image:** Grayscale image with pixel values in the range $[0, 255]$

Your function names and arguments must be exactly as follows:

```
log_transform(image)
exp_transform(image, beta)
gamma_correction(image, gamma)
```

Implementation Guide and Technical Requirements:

1. **Allowed Libraries:** For matrix calculations and working with images, you are only allowed to use `numpy` and `cv2` libraries.
2. **Image Normalization:** Before applying any mathematical transformation, you must scale the input image pixel values (initially in the range $[0, 255]$) to the range $[0, 1]$. In the relevant formulas, r represents the normalized values.
3. **Transformation Formulas:** Your functions must apply the requested transformations to the image matrix according to the following formulas:

$$s = c \cdot \log(1 + r) \quad (\text{Logarithmic transformation}) \quad (1)$$

$$s = r^\gamma \quad (\text{Gamma correction}) \quad (2)$$

4. **Output Rescaling and Casting:** After calculating the output matrix (s), you must rescale the values back to the standard image range $[0, 255]$. It is very important that the final output of your functions is of type `uint8` so that the image is displayed correctly.

1.0.3 Part 2: Analysis and Theory

In addition to implementing the code, answer the following two questions briefly and precisely:

1. **Question 1:** Suppose you have a medical image (such as a radiology X-ray) where most of the important details are located in very dark regions. Between "logarithmic transformation" and "exponential transformation", which one is more suitable for revealing details in these dark regions? Explain your reasoning based on the mathematical behavior of the function.
2. **Question 2:** An image has been captured with excessive exposure (overexposed), making it very bright; the image appears washed out and details in bright regions are lost. To correct the contrast of this image using "gamma correction", should the gamma parameter (γ) be set greater than 1 or less than 1? Why?

2 Part Two: Histogram Equalization

2.0.1 Histogram Equalization

As taught in point processing topics, inappropriate distribution of intensity values (gray levels) in an image can lead to severe contrast reduction and hidden visual details. The Histogram Equalization algorithm is one of the fundamental and important methods for automatic contrast enhancement. This method, by examining the statistical distribution of pixels and using the Cumulative Distribution Function (CDF), expands the intensity values so that the final histogram approaches a uniform distribution.

The image `pout.tif` is a classic example of an image with compressed dynamic range that has very low contrast.

You must implement the histogram equalization algorithm from scratch on this image using the taught concepts.

2.0.2 Requirements

1. **Algorithm Implementation:** Implement the histogram equalization algorithm using probability calculations and the CDF formula.
2. **Final Image Construction:** Obtain the final image from the equalized histogram.
3. **Plotting Graphs:** Create an output consisting of 4 subplots displayed together, including:
 - Original image
 - Histogram of the original image
 - Output image (after equalization)
 - Histogram of the output image
4. **Result Analysis:** In a Markdown cell in your notebook, analytically answer the following question:
"According to mathematical theory, the output histogram is expected to be completely uniform and flat; but why in practice, in digital images (like the output of your code), the final histogram is not completely uniform and has gaps and empty spaces?"

2.0.3 Submission Method

Since this exercise is evaluated through file upload, you must submit the following items in a compressed file (.zip):

1. The executed notebook file (.ipynb) containing the implementation code, display of all outputs within the notebook, and textual explanations (Markdown).
2. Output images extracted from the code (including the histogram of the original image, the equalized histogram, and the output image resulting from this histogram), which must be placed separately alongside the notebook file.

Note: The final zip file must be named exactly in the format `IP_StudentName_StudentId`.

3 Part Three: Local Filters - Noise Analysis and Evaluation

3.0.1 Local Filters

Dealing with Noise; Analysis and Evaluation of Spatial Filters

In real-world image processing systems, images often suffer from various types of noise during capture or transmission. One common type of noise is "Salt & Pepper Noise," which appears as completely black and white pixels on the image.

In this exercise, a grayscale image contaminated with Salt & Pepper noise is provided, along with the original noise-free version of the same image. Your goal is to apply different filters to remove this noise and then perform scientific and visual analysis of the results.

This question consists of two parts: Implementation and Analysis.

3.0.2 Part 1: Implementation and Execution

Write a Python program that performs the following steps:

1. Read the noisy image and the original (noise-free) image.
2. Apply the following three spatial filters with kernel size exactly 5×5 on the noisy image:
 - Mean Filter
 - Median Blur
 - Gaussian Blur
3. **Important Note:** You do not need to implement these filters from scratch. You must use the ready-made and optimized functions from the Python OpenCV library.
4. Display the resulting images from all three filters along with the original noisy image in a single window (using libraries like Matplotlib) to enable visual comparison.
5. **PSNR Evaluation Metric:** Calculate the PSNR (Peak Signal-to-Noise Ratio) between each of the three filtered images and the original noise-free image, and print the results.

Guidance: The PSNR calculation formula is as follows:

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right) \quad (3)$$

(In this formula, MAX_I is the maximum possible pixel value of the image, and MSE is the Mean Squared Error between the filtered image and the original image.)

3.0.3 Part 2: Result Analysis

After executing the code and observing the outputs, prepare a brief analytical report and answer the following questions:

1. According to the obtained PSNR values, which filter performed better in removing Salt & Pepper noise?

2. In terms of visual quality evaluation, does the visual result correlate with the PSNR values? Which filter's output better preserves edges and image details?
3. What is the scientific reason for the superiority of the winning filter against Salt & Pepper noise? Why were the other two filters unable to perform as successfully?

3.0.4 Programming Requirements

Your code must be readable and must have proper commenting and sufficient explanations for each part of the code.

3.0.5 Submission Method

Upload your answer as a compressed .zip file named `IP_StudentName_StudentId` to the system. This compressed file must exactly include the following:

1. Your code file (in .py or Jupyter notebook .ipynb format).
2. A .pdf file containing the answers to the analysis section questions and screenshots of the printed visual results.

4 Summary of Deliverables

Part	Type	Deliverables
Part One: Enhancement Functions	Implementation	<code>enhancement.py</code> with three functions
Part One: Theory Questions	Analysis	Answers to Questions 1 & 2
Part Two: Histogram Equalization	Implementation	Notebook with code, outputs, and analysis
Part Three: Local Filters	Implementation	Code file + PDF with analysis and screenshots

Table 1: Summary of required deliverables

Important Notes

- All code must be original and properly commented.
- For Part One, only `numpy` and `cv2` libraries are allowed.
- For Part Three, you may use OpenCV's built-in filter functions.
- All output images must be properly displayed and saved.
- The final zip file name must be exactly: `IP_StudentName_StudentId.zip`
- Replace "StudentName" with your actual name and "StudentId" with your actual student ID.

Grading Scheme

Component	Points
Part One - Implementation (3 functions)	6
Part One - Theory Questions (2 questions)	4
Part Two - Histogram Equalization	5
Part Three - Filters Implementation	3
Part Three - Analysis Questions	2
Total	20

Table 2: Grading scheme

Good luck!